Advanced Compilers

Project 0

Virtual Machine & Optimization Laboratory

Dept. of Electrical and Computer Engineering

Seoul National University

Index

Project 0

HEURISTIC OPTIMIZATION CONSTANT MATERIALIZATION

What is RISC-V?

RISC-V

- Open Standard instruction set architecture (ISA)

Instruction Set Architecture (ISA)

- Defines how software communicates with hardware (instructions, registers, memory model)

RISC (Reduced Instruction Set Computer)

- Simple and fixed-length instructions
  - easier decoding, better pipeline efficiency

Open & Modular Design

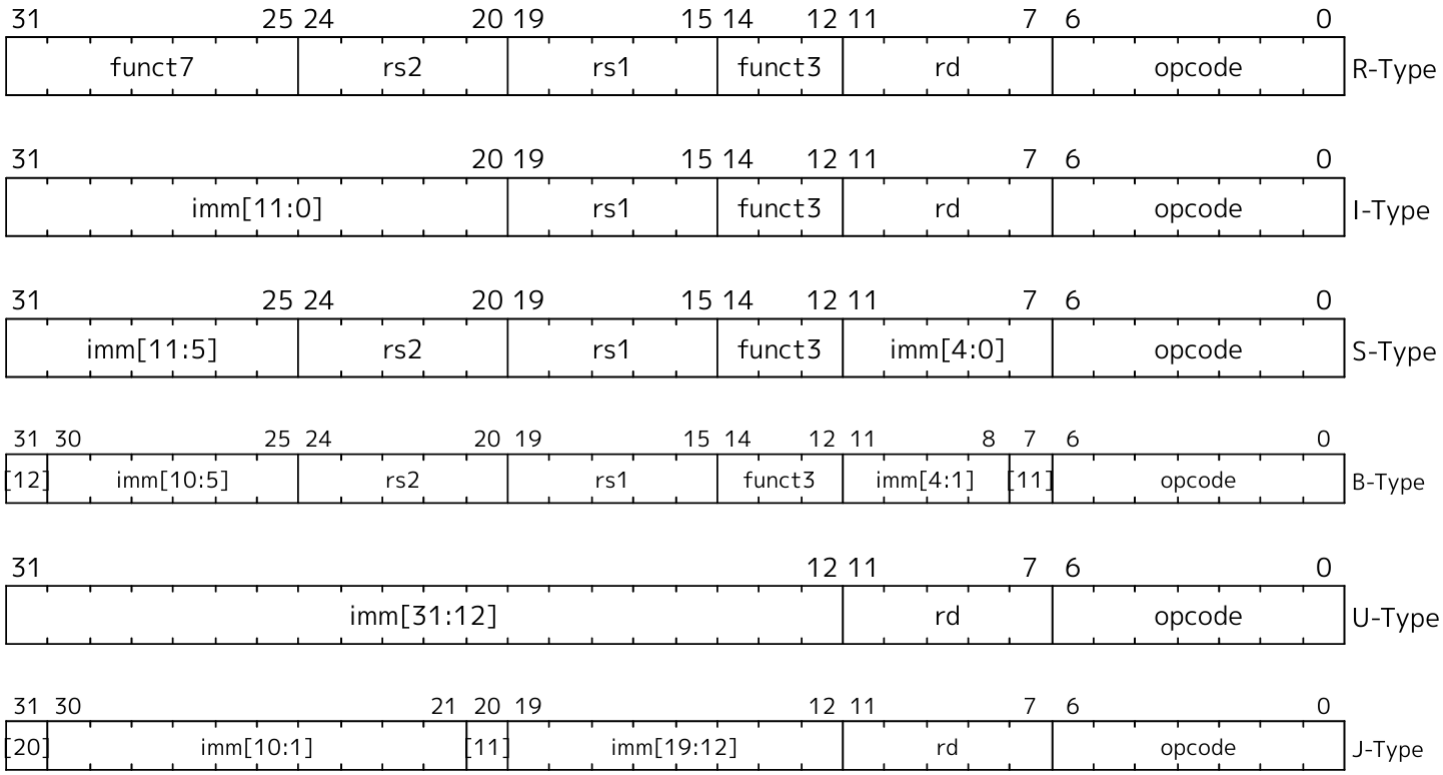
- Free to use, extensible (custom instructions possible)

Simple and regular ISA makes low-level optimization decisions explicit

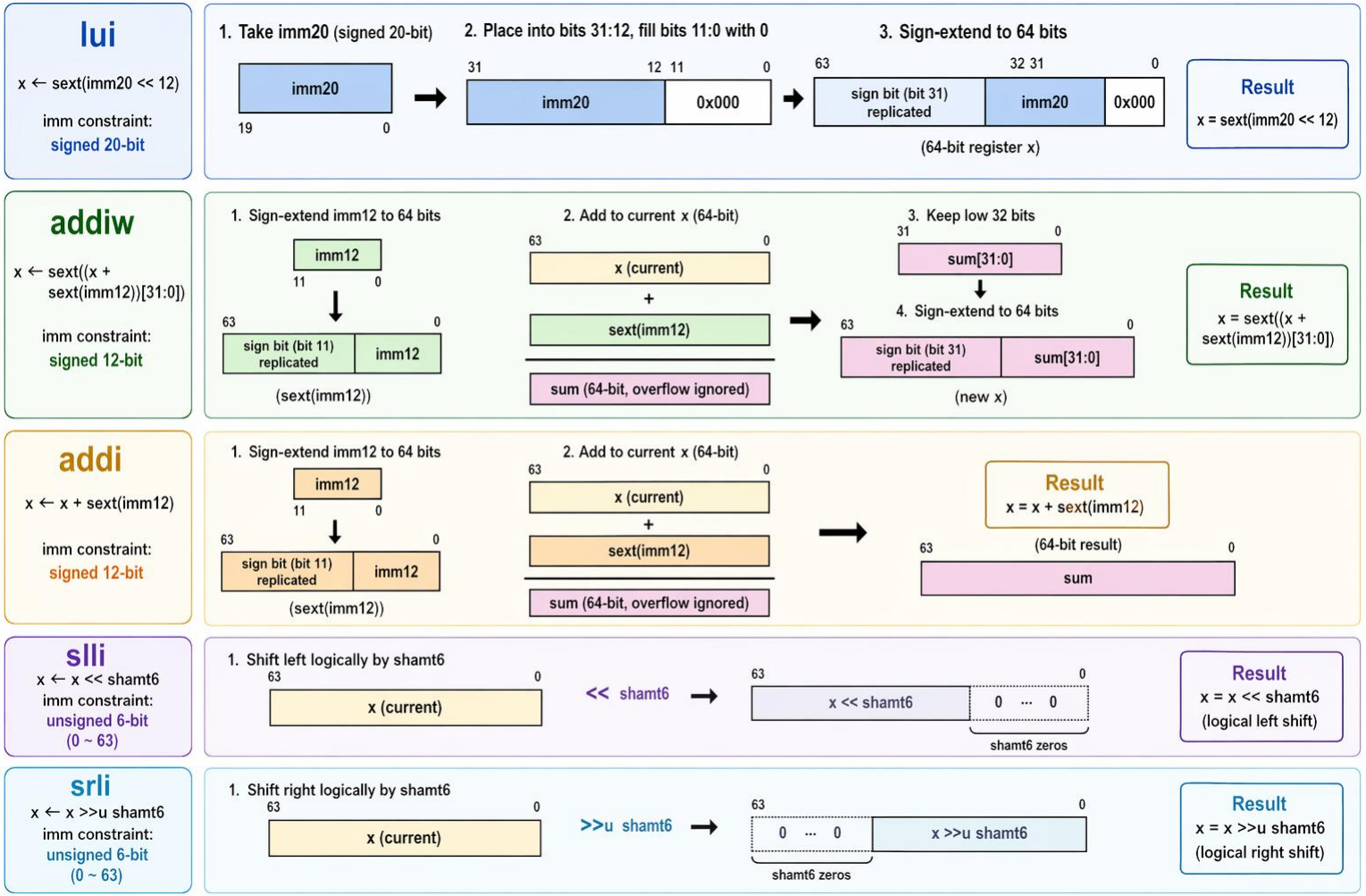
Constant Materialization in RISC-V 64

RISC-V 명령어는 32bit로 표현

- RV64에서 64 bit 상수를 register에 load 하기 위해서는 복수의 명령어가 필요



RISC-V 64 Instructions for Constant Loading



8-Instruction Baseline

[1] lui → [2] addiw → // Upper 32 Bits
[3] slli → [4] addi → // Upper 11 Bits in Lower 32 Bits
[5] slli → [6] addi → // Next 11 Bits in Lower 32 Bits
[7] slli → [8] addi // Remaining 10 Bits in Lower 32 Bits

```
[START] constant 0xCAFE'BABE'DEAD'BEEF
case: fallback 8-instruction baseline
constant: 0xCAFE'BABE'DEAD'BEEF (-3819410105021120785)
instruction count: 8
```

Step	Instruction	Reg	Target	Match
1	lui 0xC'AFEC	0xFFFF'FFFF'CAFE'C000	0xCAFE'BABE'DEAD'BEEF	mismatch
2	addiw -1346 (0xABE)	0xFFFF'FFFF'CAFE'BABE	0xCAFE'BABE'DEAD'BEEF	mismatch
3	slli 11	0xFFFF'FE57'F5D5'F000	0xCAFE'BABE'DEAD'BEEF	mismatch
4	addi 1781 (0x6F5)	0xFFFF'FE57'F5D5'F6F5	0xCAFE'BABE'DEAD'BEEF	mismatch
5	slli 11	0xFFF2'BFAE'AFB7'A800	0xCAFE'BABE'DEAD'BEEF	mismatch
6	addi 879 (0x36F)	0xFFF2'BFAE'AFB7'AB6F	0xCAFE'BABE'DEAD'BEEF	mismatch
7	slli 10	0xCAFE'BABE'DEAD'BC00	0xCAFE'BABE'DEAD'BEEF	mismatch
8	addi 751 (0x2EF)	0xCAFE'BABE'DEAD'BEEF	0xCAFE'BABE'DEAD'BEEF	match

Optimize Constant Materialization

load_constant.cpp 의 emitLoadConstant 함수 내부를 구현

- 각 Target Constants에 맞추어서 최적화된 Instruction Sequence 생성을 구현
- Baseline (8 Instructions) 보다 적은 Instruction으로 Constant Materialization을 구현하는 것이 목표

Build the project

- `make`

Run the test

- `./load_constant`

Optimization Target Constants

Zero constant

- 0x0000’0000’0000’0000 (0)

Signed 12-bit constants

- 0x0000’0000’0000’07FF (2047)
- 0xFFFF’FFFF’FFFF’F800 (-2048)
- 0x0000’0000’0000’0123 (291)

Signed 32-bit constants

- 0x0000’0000’7FFF’FFFF (2147483647)
- 0xFFFF’FFFF’8000’0000 (-2147483648)
- 0x0000’0000’1234’5678 (305419896)

Constants with at least 32 leading and trailing zeros in total

- 0x0000’0001’0000’0000 (4294967296)
- 0x0020’0000’0040’0000 (9007199258935296)
- 0x0000’0200’0000’0400 (2199023256576)

8-Instruction Baseline

```
// Build the upper 32-bit word with lui + addiw.
// upper12 is the signed 12-bit addiw immediate taken from the low 12 bits
// of that upper 32-bit word.
int64_t upper12 = static_cast<int64_t>(signExtendBits(targetBits >> 32, 12));

// addiw always adds a sign-extended 12-bit immediate.
// When upper12 is negative, that immediate is represented with leading 1s,
// so the effect is the same as subtracting from the lui result.
// Compensate for that by adding 1 to the upper 20-bit part before emitLui.
int64_t upper20 = static_cast<int64_t>(signExtendBits((targetBits >> 44) + (upper12 < 0), 20));

emitter.emitLui(upper20);
emitter.emitAddiw(upper12);

// Append the remaining low 32 bits as 11 / 11 / 10-bit chunks.
emitter.emitSlli(11);
emitter.emitAddi(static_cast<int64_t>((targetBits >> 21) & 0x7FF));

emitter.emitSlli(11);
emitter.emitAddi(static_cast<int64_t>((targetBits >> 10) & 0x7FF));

emitter.emitSlli(10);
emitter.emitAddi(static_cast<int64_t>(targetBits & 0x3FF));
```